

Training Budget and Time Parameterisation Decide the Ranking of Propagation Operators on LEO Inter-Satellite-Link Graphs

Phuc Hao Do^{a,*}

^a*CAIRA, Danang Architecture University, Da Nang, Viet Nam*

Abstract

A preliminary version of this study reported, from 3 seeds, that a propagation operator built from the transition probabilities of a continuous-time quantum walk, $|e^{-it\mathbf{L}}|^2$, outperformed the heat kernel $e^{-t\mathbf{L}}$ on a 1584-satellite low-Earth-orbit inter-satellite-link graph, and read the trend across shell sizes as a ballistic mechanism emerging with scale. This paper is a reproducibility audit of that claim. The object under test is the probability kernel, not the unitary propagator whose over-smoothing argument recent work has proved; the two kernels are both real, nonnegative and row-stochastic and differ only in row profile, which is stated before any result is read. Three operators sharing one eigenbasis and one network are compared on Walker-delta shells of 264 to 4400 satellites, on two node-field tasks, over ten seeds, across arms that vary the parameterisation and initialisation of the learned propagation time t , the capacity and the training budget, with two reach-matched decoupled baselines, for 1920 plus 720 training runs on one machine. With the preliminary budget held fixed, no operator improves on a constant predictor by more than 2.9% on any shell of 1584 satellites or more, in any arm, at any capacity; the threshold sensitivity of that statement is reported. With a five-fold budget every shell leaves the floor, and the preliminary result is not reproduced: under paired tests across seeds, no arm-level cell distinguishes the two spectral kernels at the 0.05 level after Holm correction (0 of 24), the heat kernel leads in mean in 14 cells and the walk kernel in 10, and the seed-majority verdicts the preliminary protocol would have declared move by up to 3 seeds when the metric changes from mean error to far-node error. What a learned propagation time buys is visible only against fixed-reach propagation, and only in the right arm: in its best t arm the better spectral kernel beats personalised-PageRank and power-of-adjacency baselines with no learned time by 53% at 1584 and 25% at 3168 satellites on the hop field, while in its worst arm the margin is 29% and 3%, and both kernels beat the baselines in every arm in 1 of 6 task-scale cells. The ranking at the one scale where the preliminary budget learns anything follows the t arm through a seed-level bimodality with indistinguishable escape rates for the two kernels. Long training at a fixed learning rate collapses 12.1% of runs to the constant solution, which is counted rather than dropped. The released artifact of the preliminary version is shown to have used a different configuration from its text. All code, results and the dated protocol are released.

Keywords: LEO constellations, inter-satellite links, graph neural networks, spectral propagation, quantum walks, reproducibility, evaluation methodology

*Corresponding author.

Email address: haodp@dau.edu.vn (Phuc Hao Do)

1. Introduction

The inter-satellite-link (ISL) graph of a low-Earth-orbit (LEO) mega-constellation is large and nearly regular, and its links across the seam and the polar regions come and go [1, 2]. Learned models that propagate information over this graph have been proposed for routing [3, 4] and for connectivity decisions [5]. At each layer such a model applies a propagation operator $\Phi(\mathbf{L})$ to node features and then a shared nonlinearity [6, 7], so the choice of operator is a design decision whose consequences grow with the graph.

Three operators span the design space studied here. The local step of a graph convolutional network [6], equation (1) below, moves information one hop per layer. The heat kernel $e^{-t\mathbf{L}}$ moves it diffusively, over a range that grows like $\sqrt{t}$ [8]. The transition-probability kernel of a continuous-time quantum walk [9], $|e^{-it\mathbf{L}}|^2$, is a real, row-stochastic matrix whose rows spread ballistically, over a range that grows like t , and it can be dropped into the same layer in place of the heat kernel [10, 11]. Recent work has made a separate and stronger argument for walk-based operators: the unitary propagator $e^{-it\mathbf{L}}$ applied to features damps no frequency and preserves feature norm, so it resists over-smoothing [12, 13, 14]. That argument concerns the unitary operator in feature space. It is not tested here, and we say so at the outset: the probability kernel studied in this paper is, like the heat kernel, a real nonnegative row-stochastic matrix, and it inherits from the walk only the shape of its rows (Section 3.2). What the probability kernel might still offer is longer reach at equal t , and whether that translates into a better propagation operator once t is learned is an empirical question.

A preliminary version of this study, circulated for review in June 2026 and available in the companion repository, answered that question in the affirmative: on a 1584-satellite shell the walk kernel reached a mean absolute error of 0.148 against 0.168 for the heat kernel, from 3 seeds, and the trend from smaller shells was read as the ballistic mechanism emerging with scale. This paper is a reproducibility audit of that result. It asks whether the ranking is a property of the two kernels or of the way they were evaluated, and it answers by making explicit and varying the quantities the preliminary comparison left implicit: the training budget relative to graph size, the parameterisation and initialisation of the learned propagation time t , the capacity, the number of seeds, and the inferential rule by which a winner is declared.

The contribution is the audit and what it finds.

- A protocol in which the operators share one eigendecomposition, one network, one parameter count and one training loop; in which budget, t parameterisation, t initialisation and capacity are arms; in which two reach-matched decoupled propagation baselines, personalised PageRank and a power of the normalised adjacency, separate “spectral operator” from “longer reach”; and in which every reading rule, including the paired tests and the multiplicity correction, is stated with its operating characteristics before a verdict is read (Section 4).
- Under the preliminary budget of 200 epochs, no operator improves on the constant predictor by more than 2.9% on any shell of 1584 satellites or more, in any arm, at either capacity (Section 5.1); the sensitivity of this statement to the floor threshold is tabulated. On the one shell where learning occurs the heat-versus-walk ranking changes with the t arm, through a seed-level bimodality in which the two kernels escape the constant solution at indistinguishable rates (Section 5.2).
- With a budget of 1000 epochs every shell leaves the floor and the preliminary result is not reproduced. Under paired sign and Wilcoxon tests across seeds with Holm correction, no

arm-level cell separates the two kernels at the 0.05 level (Section 5.3); the seed-majority rule the preliminary protocol used would have declared winners, and those verdicts move by up to 3 seeds when the metric changes (Section 5.4). Long training collapses 12.1% of runs to the constant solution, counted rather than dropped (Section 5.6).

- The released artifact of the preliminary version used a network of 3301 parameters while its released code defaults to 612, with no script for the experiment in question (Section 7).

Section 2 places the operators and the setting in the literature. Section 3 defines the graphs (Section 3.1), the operators (Section 3.2) and the network and tasks (Section 3.3). Section 4 states the protocol and the reading rules (Section 4.2). Section 5 reports the grids and their cost (Section 5.7). Section 6 interprets them, Section 7 lists what limits them, and Section 8 concludes.

2. Related work

Propagation operators and their reach. Spectral graph convolutions began as polynomial filters of the Laplacian [15, 7]; the graph convolutional network of Kipf and Welling [6] keeps the first-order term and applies a normalised adjacency once per layer, so depth bounds reach. Stacking layers drives features toward a constant, the over-smoothing effect identified by Li et al. [16], shown by Oono and Suzuki [17] to be exponential in depth, and surveyed by Rusch et al. [18]; remedies include decoupling propagation from transformation as in SGC [19] and personalised-PageRank propagation [20], and residual and identity terms as in GCNII [21]. Alon and Yahav [22] describe the complementary bottleneck by which information from exponentially many distant nodes is squashed into fixed-size vectors, which Topping et al. [23] relate to negative curvature of the graph. Continuous-time views replace the discrete step with a differential equation on the graph: GRAND [8] uses diffusion, whose solution is the heat kernel; PDE-GCN [24] and graph-coupled oscillator networks [25] use wave-type dynamics, whose reach grows linearly in time rather than as a square root. The distinction between diffusive and ballistic spreading that motivates the walk kernel in this paper is the same distinction, and the wave-type models are its classical form.

Quantum walks as graph operators. The continuous-time quantum walk of Farhi and Gutmann [9] evolves an amplitude vector by $e^{-it\mathbf{L}}$ and spreads ballistically; Kempe [26] and Mülken and Blumen [27] review the walk and its transport properties on complex networks, and Childs [28] shows it is computationally universal. Dernbach et al. [10] and Verdon et al. [11] were among the first to place walk-derived operators inside trainable graph models. GraphQWalk [14] builds unsupervised, position-independent structural embeddings from walk transition probabilities. CTQWformer [13] learns the walk Hamiltonian and feeds final-time probabilities into a graph transformer. CTQW-GNN [12], a preprint at the time of writing, uses the unitary propagator itself in feature space and proves that, because its eigenvalues lie on the unit circle, no frequency is damped and the Dirichlet energy of the features cannot decay exponentially with depth. That result concerns the unitary operator acting on features. The operator studied here is the transition-probability kernel $|e^{-it\mathbf{L}}|^2$, a real nonnegative matrix that shares the ballistic profile of the walk but not the norm-preserving property; the distinction matters for interpreting the results (Section 6). Quantum walks on communication networks have also been used directly, for instance the Hadamard-walk model for network identification of Liang et al. [29].

Table 1: The four shells. All use +Grid links, a 70° polar cutoff and a seam cut. The two largest are scale-ups of the Starlink shell-1 geometry and are not deployed constellations.

satellites	planes	per plane	incl. ($^\circ$)	alt. (km)	note
264	24	11	53	550	main scale of the preliminary version
1584	72	22	53	550	Starlink shell-1 geometry [2]
3168	72	44	53	550	scale-up, synthetic
4400	100	44	53	550	scale-up, synthetic

Graph learning on LEO constellations. Handley [30] first argued that inter-satellite routing in a mega-constellation is a latency question, Bhattacharjee and Singla [1] fix the +Grid inter-satellite topology that most later work assumes, Kassing et al. [31] provide the Hypatia simulator for such networks, and del Portillo et al. [2] compare the announced constellations, including the Starlink shell whose geometry the 1584-satellite graph here reproduces. Learned models on this graph include the link-identified routing architecture of Zhang et al. [3], the duality-guided graph learning for joint connectivity and routing of Gu et al. [4], and machine-learning-driven cellular-satellite multi-connectivity [5]. None of these compares propagation operators under a common parameterisation of reach, which is the gap this study addresses. The tasks used here are deliberately decoupled from any particular routing or traffic-engineering system, so that the propagation question is isolated from questions of load.

Evaluation pitfalls. Shchur et al. [32] showed that the ranking of graph neural network architectures on standard benchmarks changes with the data split and the seed, Henderson et al. [33] made the corresponding point for deep reinforcement learning, where few seeds and unreported tuning produce rankings that do not replicate, Bouthillier et al. [34] quantify how much benchmark variance comes from sources other than the method, and Pineau et al. [35] report on the reproducibility programme that turned such findings into checklists. The present study applies the same discipline to a comparison of propagation operators, and adds two quantities specific to spectral operators, the training budget relative to graph size and the parameterisation of the learned propagation time.

3. Setting

3.1. Constellation graphs

Each graph is a snapshot of a Walker-delta shell. Satellite positions follow circular-orbit propagation in an Earth-centred inertial frame; inter-satellite links follow the +Grid pattern [1], two intra-plane and two inter-plane links per satellite, with inter-plane links suppressed above 70° latitude and across the counter-rotating seam. The graph is restricted to its largest connected component. Four shells are used (Table 1). The 264-satellite shell is the main scale of the preliminary version; the 1584-satellite shell reproduces the geometry of the first Starlink shell [2], 72 planes of 22 satellites at 53° and 550 km; the 3168- and 4400-satellite shells are scale-ups of that geometry, 72 planes of 44 and 100 planes of 44, and are synthetic in the sense that no operator flies them. They exist to make the diameter grow.

Each training or evaluation instance draws a snapshot time uniformly over the orbital period and a destination satellite uniformly over the component. Instance construction is seeded, so every instance is reproducible from its index.

3.2. Three operators on one eigenbasis

Let $\mathbf{L} = \mathbf{D} - \mathbf{A}$ be the combinatorial Laplacian of the snapshot, with eigendecomposition $\mathbf{L} = \mathbf{U} \text{diag}(\lambda) \mathbf{U}^\top$ computed once per instance and shared by all operators. The three operators are

$$\Phi_{\text{gcn}} = \tilde{\mathbf{D}}^{-1/2}(\mathbf{A} + \mathbf{I})\tilde{\mathbf{D}}^{-1/2}, \quad (1)$$

$$\Phi_{\text{heat}}(t) = \mathbf{U} \text{diag}(e^{-t\lambda}) \mathbf{U}^\top = e^{-t\mathbf{L}}, \quad (2)$$

$$\Phi_{\text{walk}}(t) = \left| \mathbf{U} \text{diag}(e^{-it\lambda}) \mathbf{U}^\top \right|^2 = |e^{-it\mathbf{L}}|^2, \quad (3)$$

where the square in (3) is taken entrywise on the modulus, so that $[\Phi_{\text{walk}}]_{ij}$ is the probability that a continuous-time quantum walk started at j is found at i after time t . This choice fixes what the study can and cannot say. Both (2) and (3) are real, nonnegative and row-stochastic; applied to a feature matrix, both average neighbouring features with nonnegative weights and both damp high-frequency components. The unitary propagator $e^{-it\mathbf{L}}$ of CTQW-GNN [12] is neither of these things, and its over-smoothing argument does not transfer to (3) and is not tested here. The two kernels differ in the profile of the row as a function of graph distance, monotone for the heat kernel and oscillatory for the walk kernel, and in how that profile’s width grows with t ; given that the network trains t , it is plausible from the definitions alone that the two will be hard to tell apart, and the experiments below are the test of that expectation, not of the unitary argument.

Figure 1 makes the difference concrete on a path graph, where distance from the source is a single coordinate. A row of the heat kernel is a bell whose spatial variance grows as $t^{1.00}$; a row of the walk kernel is a pair of fronts whose variance grows as $t^{2.00}$, with oscillations between them. The exponents are fitted from the kernels themselves. This is the ballistic-versus-diffusive distinction in its cleanest form: at equal t the walk reaches farther, and its reach grows faster with t . Whether that translates into a better propagation operator once t is learned is the empirical question of this paper.

3.3. Network, parameterisation of t , and tasks

The network has K layers, each computing $\mathbf{H}^{(\ell+1)} = \sigma(\Phi^{(\ell)} \mathbf{H}^{(\ell)} \mathbf{W}^{(\ell)})$ with a ReLU nonlinearity and a linear read-out to one scalar per node. For the two spectral operators, each layer carries its own scalar propagation time t_ℓ , trained jointly with the weights. The preliminary version parameterised it as $t_\ell = \text{relu}(\theta_\ell)$ with θ_ℓ initialised at 0.5. Section 4 explains why this is an arm of the present study and adds $t_\ell = \text{softplus}(\theta_\ell)$, which keeps $t_\ell > 0$ with a live gradient, initialised so that the two parameterisations start at the same value. The GCN operator has no t .

The input to every node is two features, an indicator of the destination and the normalised degree. Two node-field targets are used. With d the destination, $h(i, d)$ the geodesic hop count and $\tau(i, d)$ the shortest-path propagation delay computed on the inter-satellite link lengths, the targets are

$$y_i^{\text{hop}} = \frac{h(i, d)}{\max_j h(j, d)}, \quad y_i^{\text{delay}} = \frac{\tau(i, d)}{\max_j \tau(j, d)}, \quad (4)$$

so that both lie in $[0, 1]$ and the constant predictor has the same meaning on every shell. The hop field is the task of the preliminary version. Both fields in (4) require the model to move a single seeded signal across the graph, which is the propagation question in isolation; neither involves traffic or load. The metric is the mean absolute error (MAE) over nodes on held-out instances.

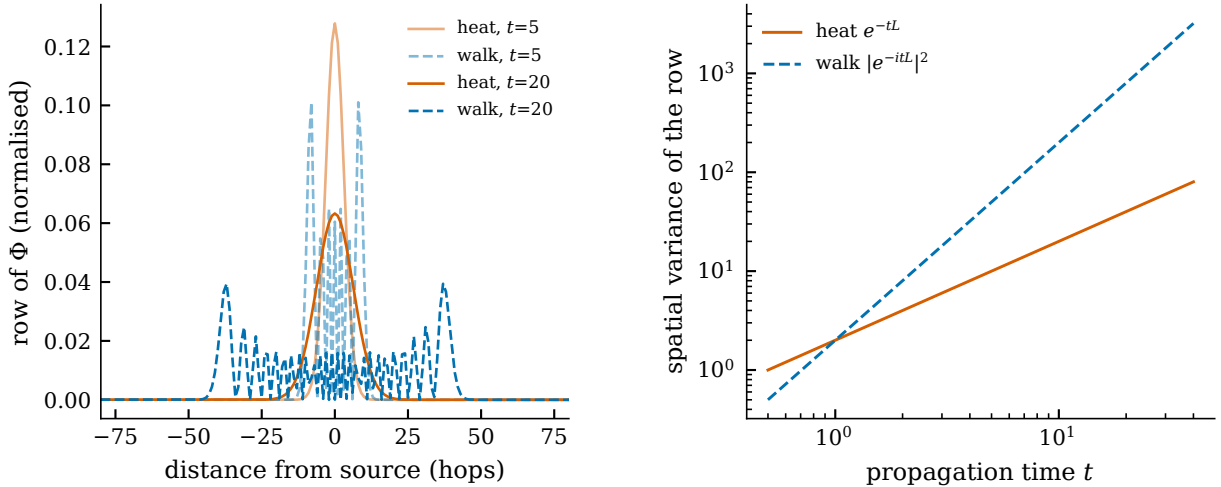


Figure 1: Analytic illustration on a path of 401 nodes. Left: one row of the heat kernel and of the walk kernel from the central node at two propagation times, normalised to sum to one. Right: spatial variance of the row against t on logarithmic axes; the slopes fitted over $t \in [7, 40]$ are 1.00 for the heat kernel and 2.00 for the walk kernel, with root-mean-square log residuals of 0.000 and 0.000.

4. Protocol

4.1. Parameter matching and the arms

The operators share depth, width, weight shapes and training loop, so a difference in error is attributable to the operator and not to capacity. Two capacities are used: width 16 with three layers, the default of the released code of the preliminary version (612 parameters including the t_ℓ), and width 32 with four layers, which the text of the preliminary version describes and its released results were produced with (3301 parameters). Adam [36] at learning rate 0.01, eight training and six evaluation instances per seed, and full-batch updates are used throughout. The six evaluation instances of a seed are drawn with that seed’s own generator, so the ten seeds of a cell evaluate on sixty distinct snapshot-destination pairs; seed-to-seed dispersion therefore includes instance-to-instance variation.

Four quantities are arms rather than fixed choices.

- **Parameterisation of t .** A pilot run under the released parameterisation showed the learned t_ℓ going negative in the second and third layers, where $\text{relu}(t_\ell) = 0$ turns the propagator into the identity and stops its gradient, so that a model reported as three or four layers of spectral propagation was propagating in one. The softplus parameterisation removes this; the relu parameterisation is kept as a control so that the preliminary configuration is one of the arms.
- **Initialisation of t .** $t_0 \in \{0.5, 2\}$, since the same pilot showed the learned reach, and with it the operator ranking, depending on where t starts.
- **Capacity.** The two configurations above.
- **Budget.** 200 epochs, the preliminary value, held fixed across scales; and 1000 epochs for the second grid.

Grid 1 runs both capacities, all four t arms, four shells, two tasks, three operators and ten seeds at 200 epochs (1920 runs). Grid 2 runs the width-32 configuration at 1000 epochs on the three smaller shells with 4 t arms (720 runs). All runs were executed on one machine, one GPU per arm, with the eigendecomposition of each instance cached in single precision and shared, so that every arm sees bit-identical eigenpairs; training already casts the eigenpairs to single precision, and on a test cell the cached and recomputed runs agreed to 16 significant digits.

Reach-matched decoupled baselines. A comparison of the three operators above confounds “spectral” with “long-reaching”: the GCN step reaches one hop per layer, the spectral kernels reach as far as their learned t allows. Two further operators, run at both budgets on the width-32 configuration, separate the two. Both decouple propagation from transformation in the sense of SGC [19] and personalised-PageRank propagation [20], have fixed reach and no learned t , and use the same network:

$$\Phi_{\text{PPR}} = \alpha \sum_{k=0}^K (1 - \alpha)^k \hat{\mathbf{A}}^k, \quad \alpha = 0.05, \quad K = 20; \quad \Phi_{\text{SGC}} = \hat{\mathbf{A}}^{K'}, \quad K' = 8, \quad (5)$$

with $\hat{\mathbf{A}}$ the normalised adjacency of (1) and Φ_{PPR} renormalised to row sums of one. Their effective reach is of the order of $1/\alpha$ and K' hops respectively, comparable to the learned reach of the spectral kernels at 1000 epochs (Section 5.3).

4.2. Reading rules, declared before the runs, with their operating characteristics

Six rules govern how a cell, one (task, shell, arm) combination, is read. The first five were fixed before the first grid was launched, and two of them were tightened after the first runs exposed a gap; both changes are dated in the repository and stated here. The sixth was added after external review of a draft showed that the fifth, as stated, had a false-positive rate the draft never computed.

1. **Constant baseline.** Every cell reports the MAE of the predictor that outputs the mean of the target. Whether an operator beats it is a result, never a filter.
2. **Pathology.** A run is excluded only for a training pathology: a non-finite loss, or a loss that rises by more than 1% between successive thirds of training. Excluded runs are counted next to every number they affect, and Section 5.6 reports whether each excluded run had left the constant solution before collapsing, since exclusion after collapse removes non-escaping runs preferentially. A run whose loss is flat from the start because it sits on the constant solution is a *plateau*, a legitimate outcome, and is kept. The first draft of this rule required a 10% loss drop and would have discarded plateaus as failures; it was corrected when the first grid produced them.
3. **Floor.** Let $\bar{e}_{\text{op}}$ be the mean MAE of an operator over its valid seeds in a cell and $\bar{e}_0$ that of the constant predictor. The cell’s gain is

$$g = \frac{\bar{e}_0 - \min_{\text{op}} \bar{e}_{\text{op}}}{\bar{e}_0}, \quad (6)$$

and the cell is *rankable* only if $g \geq 0.05$. This threshold was chosen after the first completed cells, all on the 4400-satellite shell, showed the three operators’ means within 0.002 of one another and of the constant baseline while the majority rule below still declared a winner. Because the value was set with that knowledge, every statement that depends on it is repeated at $g \in \{0.01, 0.02, 0.05, 0.10\}$ in Section 5.5.

one cell = (task, shell, arm); grid 1: 200 epochs, every arm; grid 2: 1000 epochs, submitted capacity, 4 t arms

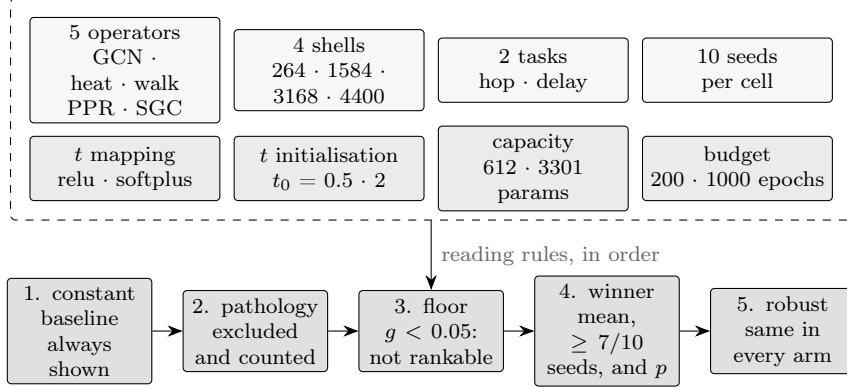


Figure 2: The study design. Every cell of the grid, one (task, shell, arm), runs the operators on the same instances and seeds; the reading rules are applied in order by one script. A majority verdict (rule 4) is reported beside the paired tests of rule 6, never instead of them.

4. **Seed majority.** Within a rankable cell, with S the set of seeds valid for both spectral operators and $e_{\text{op},s}$ the MAE of seed s , the heat kernel wins the majority if

$$\bar{e}_{\text{heat}} < \bar{e}_{\text{walk}} \quad \text{and} \quad \left| \{s \in S : e_{\text{heat},s} < e_{\text{walk},s}\} \right| \geq 7, \quad (7)$$

and symmetrically for the walk kernel; otherwise the cell is a tie, and fewer than seven valid seed pairs leaves it undecided. This is the rule of the preliminary protocol, and it is kept so that the preliminary verdicts can be reproduced. Its operating characteristics are the point: under the null of exchangeable operators, $P(X \geq 7 \mid n = 10, p = \frac{1}{2}) = 0.172$, so a winner is declared in 34.4% of null cells, and the two-sided sign-test p -values behind a 7:3, 8:2 and 8:1 majority are 0.344, 0.109 and 0.039. A majority verdict is therefore reported beside, never instead of, the tests of rule 6, and every table that prints a verdict also prints which operator led in mean, so that a “tie” cannot hide a lead.

5. **Robustness.** A ranking at a (task, shell) is claimed only if it is the same in every t arm run at that budget. Grid 2 runs 4 arms, and the number is stated wherever the rule is applied.
6. **Inference.** For every cell, the paired per-seed differences $d_s = e_{\text{heat},s} - e_{\text{walk},s}$ are summarised by their mean with a 95% bootstrap confidence interval, by the exact two-sided sign test, and by the exact two-sided Wilcoxon signed-rank test, and the Wilcoxon p -values are Holm-corrected over the 24 arm-level cells of the same metric. A difference between the two kernels is claimed only where the corrected p is below 0.05. The tests were checked against an independent implementation on random data and agree to nine decimal places.

All six rules are implemented once, in the analysis script that produces every table and figure, and the same script prints the pathology and plateau counts beside every mean. Figure 2 summarises the design, and Table 2 collects the headline numbers with the section that derives each.

5. Results

5.1. A fixed budget produces a floor at scale

Figure 3 and Table 3 show, for each capacity and each t arm, the gain (6) of the best operator over the constant predictor as the shell grows, at the preliminary budget of 200 epochs. On the

Table 2: Headline numbers and where each is derived. Every value is generated from the result files by the same script that produces the figures.

finding	value	derived in
runs, fixed 200-epoch budget (grid 1)	1920	Sec. 5.1
largest gain over constant on any shell ≥ 1584 , any arm, either capacity (grid 1)	2.9%	Fig. 3, Tab. 3
rankable cells at ≥ 1584 satellites, hop field, at floor 5% / 2% / 1% (grid 1)	0 / 2 / 5 of 24	Tab. 15
heat-vs-walk verdicts at 264 satellites across the four t arms, submitted capacity (grid 1)	tie / heat / tie / heat	Fig. 4
the same, released capacity (grid 1)	tie / tie / walk / tie	Tab. 5
runs, 1000-epoch budget, 4 t arms (grid 2)	720	Sec. 5.3
task-scale cells where heat / walk wins in every arm, MAE (grid 2)	2 / 0 of 6	Tab. 7
the same on the far-node metric (grid 2)	0 / 0 of 6	Tab. 12
arm-level cells where heat / walk leads in mean, MAE (grid 2)	14 / 10 of 24	Tab. 7
arm-level cells with Wilcoxon $p < 0.05$ before / after Holm, MAE (grid 2)	8 / 0 of 24	Tab. 9
runs collapsing to the constant solution under long training (grid 2)	87 (12.1%)	Sec. 5.6
preliminary 1584-satellite result, walk vs heat, 3 seeds	0.148 vs 0.168	Sec. 5.3

264-satellite shell the gain on the hop field is between 7.7% and 28.7% at the released capacity and between 24.9% and 40.3% at the submitted capacity; on the delay field it is between 7.9% and 19.5% and between 10.9% and 29.2%. On every larger shell it never exceeds 2.9%, in any of the eight capacity-by-arm combinations, on either task. Of the 24 hop-field cells at 1584 satellites and above, 0 are rankable at the declared 5% floor; Section 5.5 shows that 2 become rankable at a 2% floor and 5 at 1%, all at 1584 satellites, and Table 4 lists the 84 of 1436 valid large-shell runs that end more than 2% below the constant predictor: they concentrate at 1584 satellites, the walk kernel contributes 34 of them and the heat kernel 29, and the largest single-run gain among them is 18.3%. Of 1920 runs, 5 were excluded as pathologies and 48 were plateaus in the sense of rule 2, kept.

Doubling the width and adding a layer does not change this: the submitted capacity is as far below the floor at 1584 satellites as the released one. Nor does the t arm. The floor is therefore not a capacity effect and not a parameterisation effect; Section 5.3 shows that it is a budget effect.

5.2. The propagation time is a hidden variable

On the 264-satellite shell, where learning does occur at 200 epochs, the ranking between the two spectral operators is not stable. Figure 4 plots every seed for the submitted capacity under the four t arms; the majority verdicts read, in arm order, tie / heat / tie / heat. Table 5 gives the means, seed counts, the operator leading in mean, the sign-test p and the verdict for all eight capacity-by-arm combinations, including the released capacity, where they read tie / tie / walk

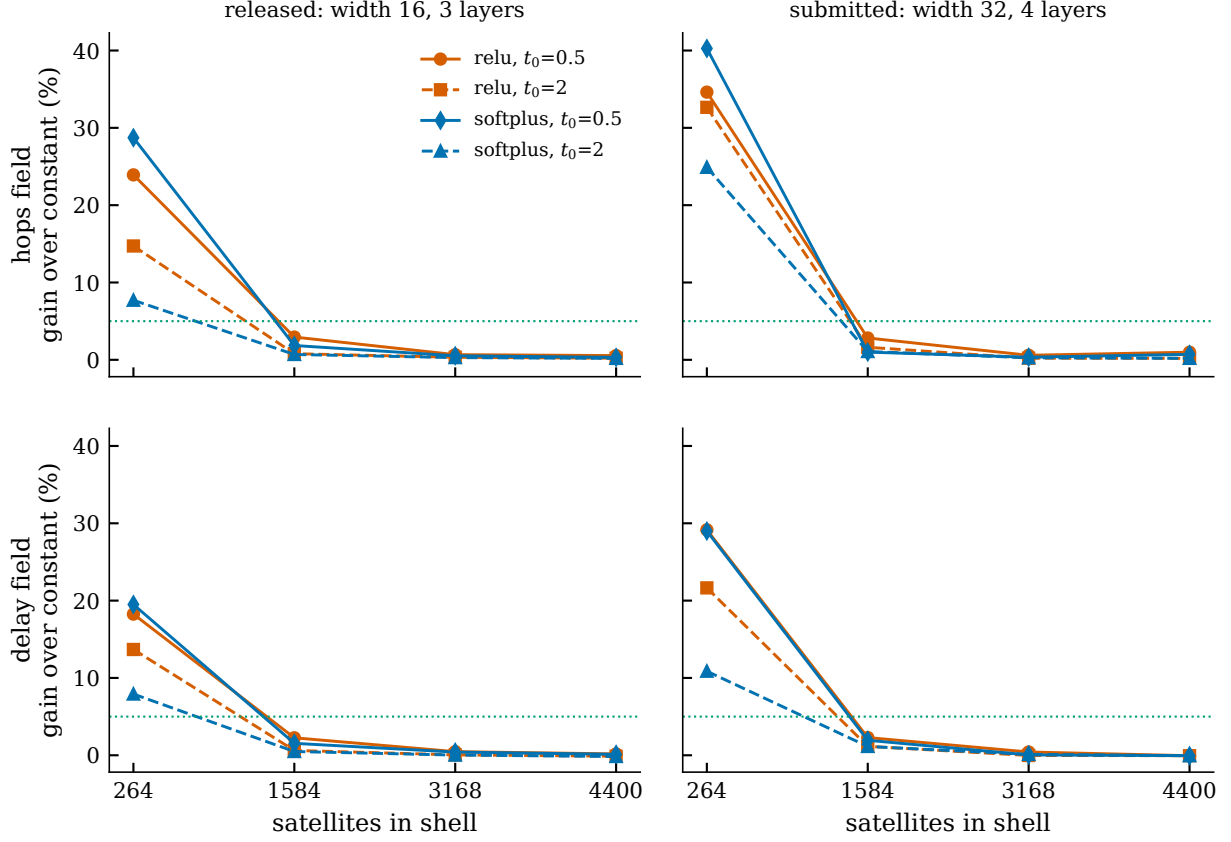


Figure 3: Gain (6) of the best operator over the constant predictor at 200 epochs, as a function of shell size, for the hop field (top) and the delay field (bottom), the released and the submitted capacity (columns) and the four t arms. The dotted line is the 5% floor of rule 3. Above 264 satellites every combination lies below it on both tasks.

/ tie. Of the 8 verdicts, 5 are ties, 2 are heat majorities and 1 is a walk majority, the latter at $p = 0.109$; no verdict repeats across all four arms at either capacity, and no cell reaches $p < 0.05$ on the sign test. The same operators on the same instances rank differently depending only on the t arm, and none of the rankings is distinguishable from noise.

The figure shows why. For both spectral operators the ten seeds do not scatter around a mean; they split. Some runs reach errors well below the constant predictor and the rest stay near it. Table 6 counts the escapes, defined as a seed whose error is below 90% of the constant predictor’s. Pooled over the eight combinations, 45/79 heat runs (57%) and 46/80 walk runs (58%) escape; a two-proportion test gives $p = 0.95$. If the escape process is a coin flip with the same bias for both operators, which is what these rates say, then every verdict in Table 5 is a difference of two or three seeds out of ten, and a mean over 3 seeds at this budget, the preliminary version’s sample, is the mean of a coin flip. The left panel of Figure 5 shows the learned t_ℓ per layer at this budget: the first layer learns a larger t than the later ones for both operators, and the values do not grow with the shell, consistent with the models on large shells not having moved from the constant solution.

Table 3: Gain (%) of the best operator over the constant predictor at 200 epochs, ten seeds per cell, both tasks. A dagger marks cells below the 5% floor, where the operator ranking is not read.

capacity	t arm	hop field				delay field			
		264	1584	3168	4400	264	1584	3168	4400
released	relu, $t_0=0.5$	23.9	2.9 [†]	0.7 [†]	0.5 [†]	18.3	2.3 [†]	0.5 [†]	0.2 [†]
	relu, $t_0=2$	14.7	0.8 [†]	0.3 [†]	0.2 [†]	13.7	0.6 [†]	0.1 [†]	-0.1 [†]
	softplus, $t_0=0.5$	28.7	1.8 [†]	0.5 [†]	0.3 [†]	19.5	1.5 [†]	0.4 [†]	0.1 [†]
	softplus, $t_0=2$	7.7	0.7 [†]	0.3 [†]	0.2 [†]	7.9	0.5 [†]	0.0 [†]	-0.2 [†]
submitted	relu, $t_0=0.5$	34.6	2.8 [†]	0.6 [†]	1.0 [†]	29.2	2.3 [†]	0.4 [†]	-0.1 [†]
	relu, $t_0=2$	32.7	1.6 [†]	0.2 [†]	0.2 [†]	21.6	1.2 [†]	-0.0 [†]	-0.1 [†]
	softplus, $t_0=0.5$	40.3	1.0 [†]	0.3 [†]	0.7 [†]	29.0	1.9 [†]	0.0 [†]	-0.1 [†]
	softplus, $t_0=2$	24.9	1.1 [†]	0.3 [†]	0.2 [†]	10.9	1.1 [†]	0.1 [†]	-0.1 [†]

Table 4: Large-shell runs at 200 epochs whose error is more than 2% below the constant predictor, by capacity, operator and shell.

capacity	operator	1584	3168	4400	all
released	GCN	8	0	0	8
	heat	15	1	2	18
	walk	19	3	2	24
submitted	GCN	0	0	0	0
	heat	14	2	0	16
	walk	15	1	2	18

Table 5: Hop field, 264 satellites, 200 epochs: MAE over valid seeds (mean $\pm$ sd for the two spectral operators, mean for the GCN, valid-seed count in parentheses; the GCN’s sd is 0.012 or less in every cell) under each capacity and t arm, the number of seeds on which each spectral operator is lower, the two-sided sign-test p , and the majority verdict. The constant predictor scores 0.189.

capacity	t arm	GCN	heat	walk	heat:walk	p_{sign}	verdict
released	relu, $t_0=0.5$	0.175 (10)	0.144 $\pm$ 0.040 (10)	0.145 $\pm$ 0.044 (10)	6:4	0.754	tie
	relu, $t_0=2$	0.175 (10)	0.161 $\pm$ 0.034 (10)	0.163 $\pm$ 0.034 (10)	6:4	0.754	tie
	softplus, $t_0=0.5$	0.175 (10)	0.161 $\pm$ 0.032 (10)	0.135 $\pm$ 0.050 (10)	2:8	0.109	walk
	softplus, $t_0=2$	0.175 (10)	0.176 $\pm$ 0.020 (10)	0.174 $\pm$ 0.022 (10)	4:6	0.754	tie
submitted	relu, $t_0=0.5$	0.166 (10)	0.124 $\pm$ 0.035 (10)	0.139 $\pm$ 0.042 (10)	6:4	0.754	tie
	relu, $t_0=2$	0.166 (10)	0.127 $\pm$ 0.045 (9)	0.152 $\pm$ 0.033 (10)	8:1	0.039	heat
	softplus, $t_0=0.5$	0.166 (10)	0.137 $\pm$ 0.031 (10)	0.113 $\pm$ 0.045 (10)	4:6	0.754	tie
	softplus, $t_0=2$	0.166 (10)	0.142 $\pm$ 0.042 (10)	0.158 $\pm$ 0.033 (10)	7:3	0.344	heat

5.3. With enough budget, the two kernels are not distinguishable

Grid 2 raises the budget to 1000 epochs at the submitted capacity on the three smaller shells, with 4 t arms. Every cell leaves the floor: on the hop field the best operator improves on the constant predictor by 13% to 70% depending on shell and arm, and on the delay field by 13% to 36% (Table 7). The floor of Section 5.1 was a budget floor. Table 8 and the right panel of Figure 5

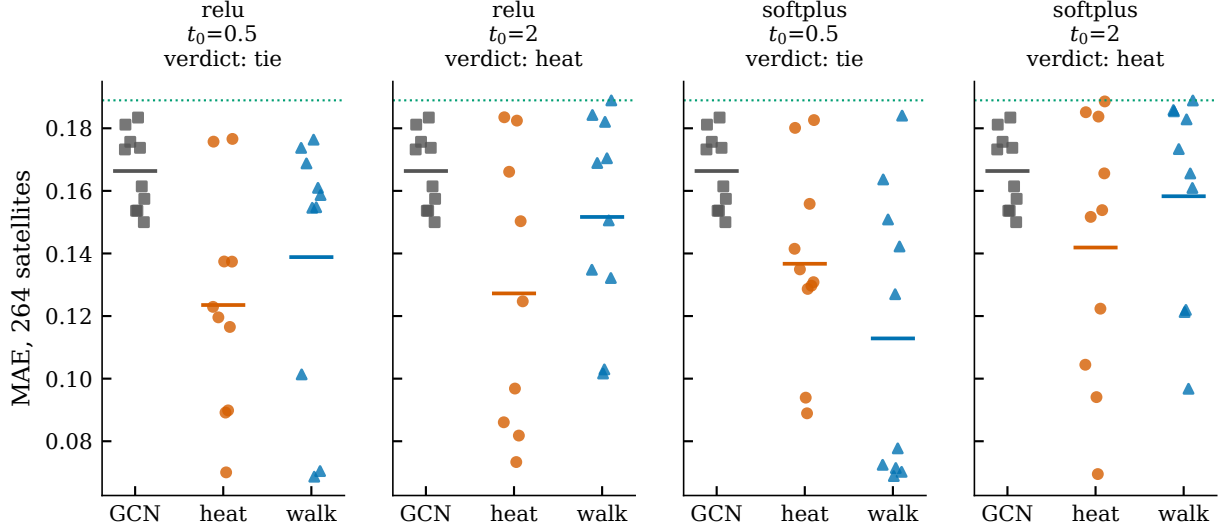


Figure 4: Per-seed MAE on the hop field, 264 satellites, submitted capacity, 200 epochs, for the four t arms. Bars are means over valid seeds; the dotted line is the constant predictor. The two spectral operators are bimodal across seeds: a run either leaves the constant solution or stays near it.

Table 6: Seeds that escaped the constant solution (MAE below 90% of the constant predictor) out of valid seeds, hop field, 264 satellites, 200 epochs.

capacity	t arm	heat: escaped/valid	walk: escaped/valid
released	relu, $t_0=0.5$	5/10	6/10
	relu, $t_0=2$	4/10	4/10
	softplus, $t_0=0.5$	4/10	6/10
	softplus, $t_0=2$	2/10	2/10
submitted	relu, $t_0=0.5$	8/10	8/10
	relu, $t_0=2$	7/9	6/10
	softplus, $t_0=0.5$	8/10	9/10
	softplus, $t_0=2$	7/10	5/10

show what the budget buys: the heat kernel’s first-layer t grows with the shell, from 3.1 at 264 satellites to 8.2 at 1584 and 10.6 at 3168, while the walk kernel’s stays between 1.3 and 2.3. The diffusive operator reaches farther on larger graphs by learning to.

Figure 6 and Tables 7 and 9 give the comparison. Read by the majority rule (7) of the preliminary protocol, the heat kernel wins in every arm at 2 of 6 task-scale cells and the walk kernel at 0. Read by mean, the heat kernel leads in 14 and the walk kernel in 10 of the 24 arm-level cells, 7 of the walk leads being recorded as ties by the majority rule. Read by inference, which is the reading this paper stands on, 8 of 24 cells reach $p < 0.05$ on the Wilcoxon test before correction and 0 after Holm correction; the sign-test p behind the heat majorities that survive rule 5 ranges from 0.008 to 0.344. No difference between the two kernels is claimed at any scale. The ratio of the walk kernel’s seed standard deviation to the heat kernel’s has median 1.40 over the 24 cells, exceeds 3 in 2 and is below 1 in 6; at ten seeds the relative standard error of a standard deviation is about 24%, so no

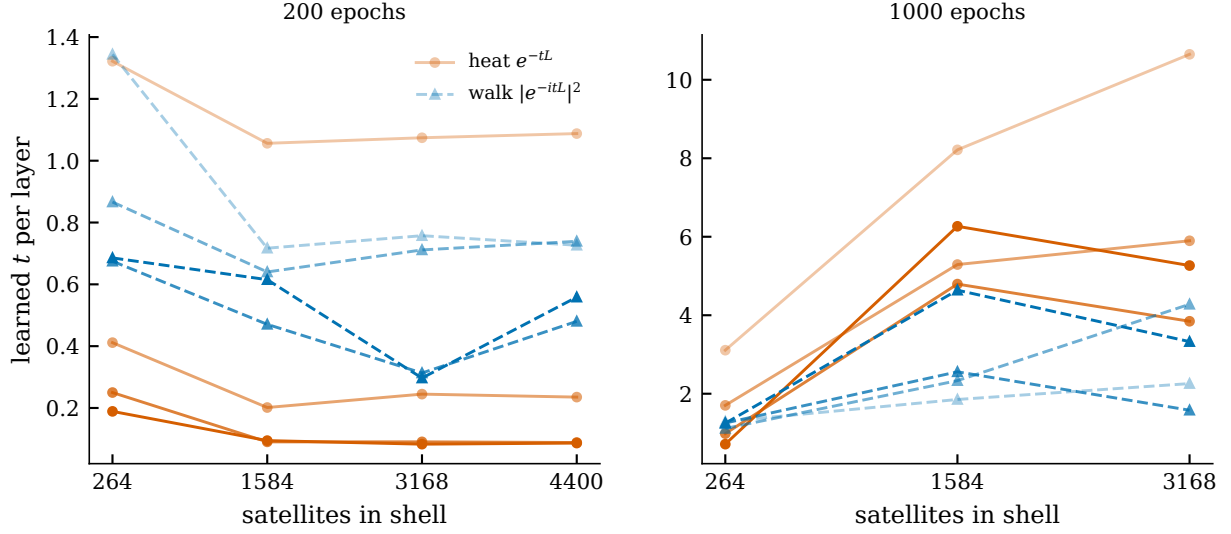


Figure 5: Learned propagation time per layer (lighter to darker: first to fourth layer) against shell size, hop field, submitted capacity, softplus arm with $t_0 = 0.5$, mean over valid seeds. Left: 200 epochs (grid 1). Right: 1000 epochs (grid 2). Note the different vertical scales.

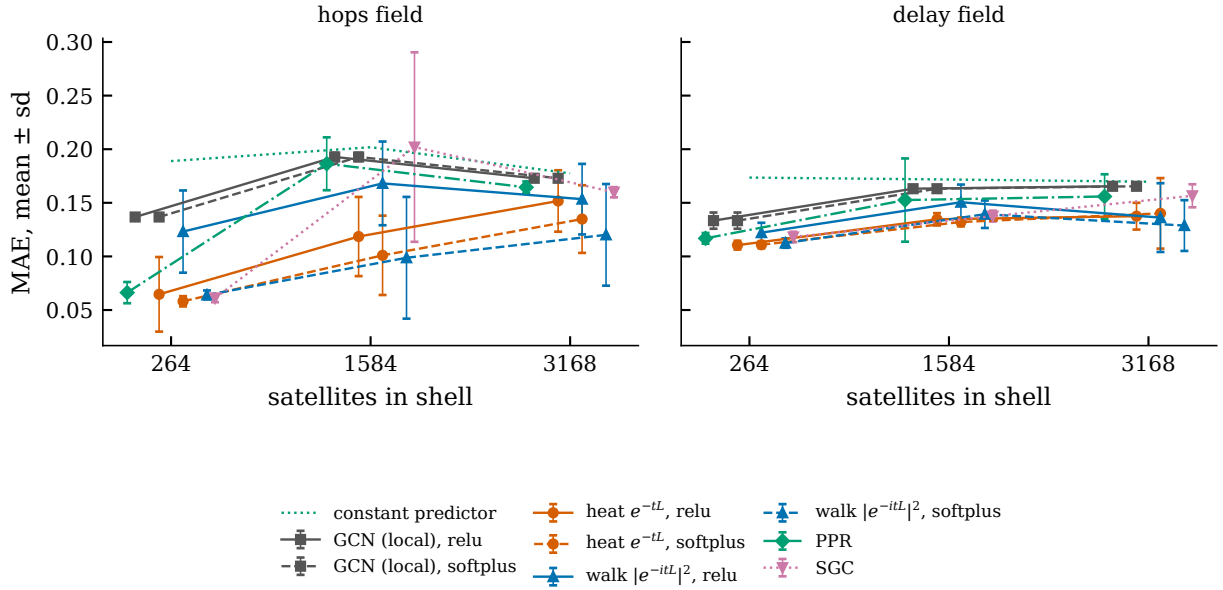


Figure 6: MAE against shell size at 1000 epochs, submitted capacity, the two $t_0 = 0.5$ arms, both tasks, with the reach-matched PPR and SGC baselines where run. Points are means over valid seeds and bars one standard deviation; the dotted line is the constant predictor. Points are offset horizontally within each shell for legibility.

statement about relative variance is made either. The preliminary version’s finding, a walk-kernel error of 0.148 against 0.168 at 1584 satellites, is not reproduced: under the same parameterisation and capacity, with ten seeds instead of 3, the two kernels are indistinguishable at that scale on both tasks.

Table 7: Grid 2, MAE at 1000 epochs, submitted capacity: mean $\pm$ sd over valid seeds with the count in parentheses, the operator leading in mean, the number of seeds on which each spectral operator is lower, and the majority verdict of rule 4. “n<7” marks a cell with too few valid seed pairs for that rule.

task, t arm	N	GCN	heat	walk	leader	heat:walk	verdict
<i>hops field, relu, $t_0=0.5$</i>							
	264	0.137 $\pm$ 0.002 (8)	0.065 $\pm$ 0.035 (9)	0.123 $\pm$ 0.038 (10)	heat	8:1	heat
	1584	0.193 $\pm$ 0.004 (9)	0.119 $\pm$ 0.037 (6)	0.168 $\pm$ 0.039 (10)	heat	4:2	n<7
	3168	0.173 $\pm$ 0.002 (9)	0.152 $\pm$ 0.029 (9)	0.153 $\pm$ 0.033 (9)	walk	5:3	tie
<i>hops field, relu, $t_0=2$</i>							
	264	0.137 $\pm$ 0.002 (8)	0.057 $\pm$ 0.002 (9)	0.093 $\pm$ 0.037 (9)	heat	8:0	heat
	1584	0.193 $\pm$ 0.004 (9)	0.132 $\pm$ 0.037 (8)	0.107 $\pm$ 0.062 (8)	walk	3:4	tie
	3168	0.173 $\pm$ 0.002 (9)	0.161 $\pm$ 0.028 (7)	0.154 $\pm$ 0.052 (5)	heat	3:1	n<7
<i>hops field, softplus, $t_0=0.5$</i>							
	264	0.137 $\pm$ 0.002 (8)	0.058 $\pm$ 0.005 (10)	0.064 $\pm$ 0.004 (10)	heat	7:3	heat
	1584	0.193 $\pm$ 0.004 (9)	0.101 $\pm$ 0.037 (10)	0.099 $\pm$ 0.057 (8)	walk	2:6	tie
	3168	0.173 $\pm$ 0.002 (9)	0.135 $\pm$ 0.032 (7)	0.120 $\pm$ 0.047 (8)	walk	2:3	n<7
<i>hops field, softplus, $t_0=2$</i>							
	264	0.137 $\pm$ 0.002 (8)	0.056 $\pm$ 0.004 (9)	0.066 $\pm$ 0.005 (9)	heat	7:1	heat
	1584	0.193 $\pm$ 0.004 (9)	0.102 $\pm$ 0.047 (7)	0.089 $\pm$ 0.041 (8)	walk	2:4	n<7
	3168	0.173 $\pm$ 0.002 (9)	0.135 $\pm$ 0.034 (8)	0.120 $\pm$ 0.051 (9)	walk	5:2	tie
<i>delay field, relu, $t_0=0.5$</i>							
	264	0.133 $\pm$ 0.008 (8)	0.111 $\pm$ 0.004 (10)	0.122 $\pm$ 0.009 (7)	heat	7:0	heat
	1584	0.163 $\pm$ 0.003 (9)	0.135 $\pm$ 0.006 (10)	0.151 $\pm$ 0.016 (10)	heat	8:2	heat
	3168	0.165 $\pm$ 0.003 (8)	0.138 $\pm$ 0.012 (10)	0.136 $\pm$ 0.032 (10)	walk	6:4	tie
<i>delay field, relu, $t_0=2$</i>							
	264	0.133 $\pm$ 0.008 (8)	0.111 $\pm$ 0.004 (10)	0.121 $\pm$ 0.012 (8)	heat	7:1	heat
	1584	0.163 $\pm$ 0.003 (9)	0.134 $\pm$ 0.004 (10)	0.137 $\pm$ 0.011 (10)	heat	8:2	heat
	3168	0.165 $\pm$ 0.003 (8)	0.159 $\pm$ 0.049 (9)	0.148 $\pm$ 0.029 (10)	walk	4:5	tie
<i>delay field, softplus, $t_0=0.5$</i>							
	264	0.133 $\pm$ 0.008 (8)	0.111 $\pm$ 0.003 (9)	0.112 $\pm$ 0.004 (10)	heat	7:2	heat
	1584	0.163 $\pm$ 0.003 (9)	0.132 $\pm$ 0.004 (10)	0.139 $\pm$ 0.013 (10)	heat	8:2	heat
	3168	0.165 $\pm$ 0.003 (8)	0.140 $\pm$ 0.033 (9)	0.129 $\pm$ 0.024 (7)	walk	1:5	n<7
<i>delay field, softplus, $t_0=2$</i>							
	264	0.133 $\pm$ 0.008 (8)	0.111 $\pm$ 0.005 (10)	0.113 $\pm$ 0.005 (10)	heat	9:1	heat
	1584	0.163 $\pm$ 0.003 (9)	0.135 $\pm$ 0.005 (10)	0.134 $\pm$ 0.005 (9)	heat	4:5	tie
	3168	0.165 $\pm$ 0.003 (8)	0.134 $\pm$ 0.025 (10)	0.116 $\pm$ 0.024 (9)	walk	3:6	tie

Table 8: Learned propagation time per layer at 1000 epochs, hop field, submitted capacity, softplus arm with $t_0 = 0.5$, mean over valid seeds.

N	heat $e^{-t\mathbf{L}}$				walk $ e^{-it\mathbf{L}} ^2$			
	t_1	t_2	t_3	t_4	t_1	t_2	t_3	t_4
264	3.1	1.7	1.0	0.7	1.3	1.1	1.3	1.2
1584	8.2	5.3	4.8	6.3	1.9	2.3	2.6	4.6
3168	10.6	5.9	3.8	5.3	2.3	4.3	1.6	3.3

The local GCN operator is the worst everywhere, as expected for a four-hop model on graphs whose eccentricity is tens of hops; on the hop field it clears the 5% floor only on the 264-satellite shell (gain 27.6%, against 4.5% at 1584 and 2.6% at 3168). Table 10 places training error beside test error for each operator, as a reference for what each model achieves on the instances it was

Table 9: Grid 2, MAE: paired inference across seeds for each arm-level cell. Mean of the per-seed difference $e_{\text{heat}} - e_{\text{walk}}$ (negative favours the heat kernel) with 95% bootstrap interval, exact two-sided sign-test p , exact two-sided Wilcoxon signed-rank p , and the Wilcoxon p Holm-corrected over the 24 cells.

task, t arm	N	$\bar{e}_{\text{heat}} - \bar{e}_{\text{walk}}$	95% CI	p_{sign}	p_{Wilcoxon}	p_{Holm}
<i>hops field, relu, $t_0=0.5$</i>						
<i>hops field, relu, $t_0=2$</i>						
<i>hops field, softplus, $t_0=0.5$</i>						
<i>hops field, softplus, $t_0=2$</i>						
<i>delay field, relu, $t_0=0.5$</i>						
<i>delay field, relu, $t_0=2$</i>						
<i>delay field, softplus, $t_0=0.5$</i>						
<i>delay field, softplus, $t_0=2$</i>						
	264	-0.0548	[-0.0793, -0.0294]	0.039	0.008	0.180
	1584	-0.0341	[-0.0808, +0.0163]	0.688	0.312	1.000
	3168	+0.0035	[-0.0228, +0.0310]	0.727	0.844	1.000
	264	-0.0384	[-0.0644, -0.0144]	0.008	0.008	0.180
	1584	+0.0286	[-0.0381, +0.0890]	1.000	0.578	1.000
	3168	-0.0053	[-0.0444, +0.0434]	0.625	0.875	1.000
	264	-0.0059	[-0.0103, -0.0009]	0.344	0.049	0.830
	1584	+0.0076	[-0.0392, +0.0486]	0.289	0.641	1.000
	3168	+0.0133	[-0.0349, +0.0541]	1.000	0.812	1.000
	264	-0.0089	[-0.0136, -0.0043]	0.070	0.016	0.328
	1584	+0.0060	[-0.0277, +0.0350]	0.688	0.688	1.000
	3168	+0.0070	[-0.0247, +0.0474]	0.453	0.938	1.000
	264	-0.0130	[-0.0184, -0.0074]	0.016	0.016	0.328
	1584	-0.0159	[-0.0248, -0.0075]	0.109	0.020	0.371
	3168	+0.0013	[-0.0162, +0.0205]	0.754	0.922	1.000
	264	-0.0112	[-0.0195, -0.0033]	0.070	0.055	0.875
	1584	-0.0037	[-0.0092, +0.0006]	0.109	0.105	1.000
	3168	+0.0144	[-0.0153, +0.0497]	1.000	0.652	1.000
	264	-0.0010	[-0.0020, +0.0001]	0.180	0.129	1.000
	1584	-0.0072	[-0.0149, -0.0014]	0.109	0.037	0.668
	3168	+0.0255	[+0.0032, +0.0528]	0.219	0.156	1.000
	264	-0.0016	[-0.0022, -0.0009]	0.021	0.004	0.094
	1584	-0.0004	[-0.0037, +0.0034]	1.000	1.000	1.000
	3168	+0.0138	[-0.0095, +0.0356]	0.508	0.203	1.000

fitted to: the spectral kernels fit their training instances to errors well below their test errors on the small shell and closer to them on the large shells, and the GCN fits neither.

Reach-matched baselines. Table 11 reports the two decoupled operators of (5) beside the better of the two spectral kernels in each cell. They carry no learned t and reach a fixed number of hops, so a spectral kernel that beats them does so by something other than reach alone. At 200 epochs the fixed-reach operators sit on the same floor as everything else on the large shells, gaining at most 6.5% over the constant predictor there. At 1000 epochs the comparison separates: on the 264-satellite hop field the better fixed-reach operator reaches 0.061 against 0.056 for the better spectral kernel, a gap of 8%, but at 1584 satellites it reaches 0.186 against 0.089 (53%) and at 3168 satellites 0.160 against 0.120 (25%); on the delay field the gaps are 5%, 4% and 25%. In the worst t arm, however, the margin shrinks to 29% at 1584 and 3% at 3168 satellites, and both kernels beat both baselines in every arm in only 1 of the six task-scale cells. A learned propagation time can therefore earn its cost on the larger graphs, where a fixed reach of $1/\alpha$ or K' hops is short of

Table 10: Training and test MAE at 1000 epochs, submitted capacity, softplus arm with $t_0 = 0.5$, mean over valid seeds. The training column is the error on the eight instances each model was fitted to and serves as the achievable-error reference for that model.

task	N	GCN		heat		walk	
		train	test	train	test	train	test
hops	264	0.137	0.137	0.058	0.058	0.064	0.064
hops	1584	0.193	0.193	0.101	0.101	0.099	0.099
hops	3168	0.173	0.173	0.135	0.135	0.120	0.120
delay	264	0.133	0.133	0.112	0.111	0.112	0.112
delay	1584	0.163	0.163	0.132	0.132	0.140	0.139
delay	3168	0.165	0.165	0.140	0.140	0.129	0.129

Table 11: Reach-matched decoupled baselines: MAE (mean $\pm$ sd over valid seeds; at 200 epochs all ten seeds are valid, at 1000 epochs the count is in parentheses) of personalised-PageRank propagation and of the K' -th power of the normalised adjacency, at 200 and 1000 epochs, submitted capacity, beside the lower of the two spectral kernels' means at 1000 epochs.

task	N	200 epochs		1000 epochs		best of heat/walk
		PPR	SGC	PPR	SGC	
hops	264	0.161 $\pm$ 0.040	0.161 $\pm$ 0.038	0.066 $\pm$ 0.010 (8)	0.061 $\pm$ 0.004 (9)	0.056
hops	1584	0.200 $\pm$ 0.004	0.201 $\pm$ 0.001	0.186 $\pm$ 0.025 (7)	0.202 $\pm$ 0.088 (7)	0.089
hops	3168	0.177 $\pm$ 0.000	0.177 $\pm$ 0.001	0.164 $\pm$ 0.005 (9)	0.160 $\pm$ 0.005 (8)	0.120
hops	4400	0.187 $\pm$ 0.000	0.187 $\pm$ 0.000	—	—	—
delay	264	0.170 $\pm$ 0.009	0.172 $\pm$ 0.004	0.117 $\pm$ 0.005 (10)	0.118 $\pm$ 0.005 (10)	0.111
delay	1584	0.172 $\pm$ 0.003	0.171 $\pm$ 0.003	0.153 $\pm$ 0.039 (8)	0.138 $\pm$ 0.005 (10)	0.132
delay	3168	0.170 $\pm$ 0.003	0.170 $\pm$ 0.003	0.156 $\pm$ 0.021 (10)	0.157 $\pm$ 0.011 (9)	0.116
delay	4400	0.169 $\pm$ 0.002	0.169 $\pm$ 0.002	—	—	—

the eccentricity, but whether it does depends on how t is parameterised and initialised, which is the paper's second finding seen from the other side. What survives is a statement about the family under a good t arm, not about which member of the family is used.

5.4. The majority verdict moves with the metric

The preliminary version also reported the area under the error curve restricted to far nodes, those beyond half the destination's eccentricity, as the metric on which a longer reach should show most clearly. Formally, with $h(i, d)$ the hop distance of node i from the destination and ϵ the destination's eccentricity, the far-node error is the mean absolute error over $\{i : h(i, d) > \epsilon/2\}$; it is in the same normalised units as the MAE and is larger because the far nodes are the hard ones. Table 12 reports it for grid 2 under the same rules, with paired inference in Table 13, and Table 14 sets the seed votes on the two metrics side by side for every cell.

On the far-node metric the heat kernel wins in every arm at 0 of 6 task-scale cells, against 2 on MAE; 4 cells reach $p < 0.05$ before correction and 0 after. The same runs, scored on the two metrics, move the seed vote by a median of 1 and up to 3 seeds, and by two or more in 9 of 24 cells. That is the noise amplitude of the majority statistic, measured on the data themselves, and it is the size of the margins by which the majority verdicts of Tables 5 and 7 were declared. On

Table 12: Grid 2, far-node error at 1000 epochs, submitted capacity: mean $\pm$ sd over valid seeds with the count in parentheses, the operator leading in mean, the seed votes, and the majority verdict of rule 4 applied to this metric.

task, t arm	N	GCN	heat	walk	leader	heat:walk	verdict
<i>hops field, relu, $t_0=0.5$</i>							
	264	0.211 $\pm$ 0.003 (8)	0.109 $\pm$ 0.066 (9)	0.186 $\pm$ 0.057 (10)	heat	7:2	heat
	1584	0.257 $\pm$ 0.016 (9)	0.198 $\pm$ 0.063 (6)	0.227 $\pm$ 0.049 (10)	heat	3:3	floor
	3168	0.243 $\pm$ 0.005 (9)	0.236 $\pm$ 0.051 (9)	0.220 $\pm$ 0.043 (9)	walk	2:6	floor
<i>hops field, relu, $t_0=2$</i>							
	264	0.211 $\pm$ 0.003 (8)	0.083 $\pm$ 0.012 (9)	0.139 $\pm$ 0.054 (9)	heat	8:0	heat
	1584	0.257 $\pm$ 0.016 (9)	0.203 $\pm$ 0.033 (8)	0.142 $\pm$ 0.079 (8)	walk	2:5	tie
	3168	0.243 $\pm$ 0.005 (9)	0.230 $\pm$ 0.028 (7)	0.209 $\pm$ 0.057 (5)	walk	1:3	floor
<i>hops field, softplus, $t_0=0.5$</i>							
	264	0.211 $\pm$ 0.003 (8)	0.090 $\pm$ 0.010 (10)	0.100 $\pm$ 0.008 (10)	heat	6:4	tie
	1584	0.257 $\pm$ 0.016 (9)	0.149 $\pm$ 0.035 (10)	0.134 $\pm$ 0.063 (8)	walk	2:6	tie
	3168	0.243 $\pm$ 0.005 (9)	0.200 $\pm$ 0.040 (7)	0.180 $\pm$ 0.069 (8)	walk	2:3	floor
<i>hops field, softplus, $t_0=2$</i>							
	264	0.211 $\pm$ 0.003 (8)	0.093 $\pm$ 0.008 (9)	0.105 $\pm$ 0.012 (9)	heat	5:3	tie
	1584	0.257 $\pm$ 0.016 (9)	0.158 $\pm$ 0.041 (7)	0.112 $\pm$ 0.055 (8)	walk	2:4	n<7
	3168	0.243 $\pm$ 0.005 (9)	0.216 $\pm$ 0.048 (8)	0.187 $\pm$ 0.056 (9)	walk	2:5	floor
<i>delay field, relu, $t_0=0.5$</i>							
	264	0.189 $\pm$ 0.010 (8)	0.152 $\pm$ 0.009 (10)	0.178 $\pm$ 0.017 (7)	heat	7:0	heat
	1584	0.209 $\pm$ 0.002 (9)	0.183 $\pm$ 0.014 (10)	0.197 $\pm$ 0.020 (10)	heat	6:4	floor
	3168	0.226 $\pm$ 0.004 (8)	0.197 $\pm$ 0.015 (10)	0.183 $\pm$ 0.036 (10)	walk	5:5	floor
<i>delay field, relu, $t_0=2$</i>							
	264	0.189 $\pm$ 0.010 (8)	0.150 $\pm$ 0.011 (10)	0.176 $\pm$ 0.020 (8)	heat	6:2	tie
	1584	0.209 $\pm$ 0.002 (9)	0.180 $\pm$ 0.009 (10)	0.180 $\pm$ 0.012 (10)	walk	6:4	floor
	3168	0.226 $\pm$ 0.004 (8)	0.209 $\pm$ 0.013 (9)	0.186 $\pm$ 0.039 (10)	walk	4:5	floor
<i>delay field, softplus, $t_0=0.5$</i>							
	264	0.189 $\pm$ 0.010 (8)	0.156 $\pm$ 0.007 (9)	0.160 $\pm$ 0.006 (10)	heat	5:4	tie
	1584	0.209 $\pm$ 0.002 (9)	0.169 $\pm$ 0.006 (10)	0.178 $\pm$ 0.021 (10)	heat	7:3	floor
	3168	0.226 $\pm$ 0.004 (8)	0.190 $\pm$ 0.053 (9)	0.187 $\pm$ 0.023 (7)	walk	3:3	floor
<i>delay field, softplus, $t_0=2$</i>							
	264	0.189 $\pm$ 0.010 (8)	0.153 $\pm$ 0.012 (10)	0.161 $\pm$ 0.006 (10)	heat	8:2	heat
	1584	0.209 $\pm$ 0.002 (9)	0.172 $\pm$ 0.008 (10)	0.176 $\pm$ 0.010 (9)	heat	5:4	floor
	3168	0.226 $\pm$ 0.004 (8)	0.181 $\pm$ 0.024 (10)	0.177 $\pm$ 0.034 (9)	heat	5:4	floor

the metric chosen to favour reach, the operator built for reach does not win; but neither, under inference, does the other.

5.5. Sensitivity of the reading rules

Table 15 repeats the two rule-dependent counts of this paper at other values of the rules: the number of task-scale cells in which each kernel wins in every arm under the majority rule, at floors of 1%, 2%, 5% and 10% and at majorities of six, seven and eight of ten; and the number of large-shell hop-field cells of grid 1 that are rankable at each floor. The statement that no operator learns on the large shells at 200 epochs holds at 5% and 10%, weakens to 2 of 24 cells at 2% and 5 at 1%, all on the 1584-satellite shell; the inference of Section 5.3 does not depend on the floor.

5.6. Long training collapses a tenth of the runs

Of the 720 runs of grid 2, 87 (12.1%) were excluded under rule 2. Inspection shows one pattern: the loss falls, then rises again by up to 88%, and the run ends on the constant solution. Table 16

Table 13: Grid 2, far-node error: paired inference across seeds, as in Table 9.

task, t arm	N	$\bar{e}_{\text{heat}} - \bar{e}_{\text{walk}}$	95% CI	p_{sign}	p_{Wilcoxon}	p_{Holm}
<i>hops field, relu, $t_0=0.5$</i>						
<i>hops field, relu, $t_0=2$</i>						
<i>hops field, softplus, $t_0=0.5$</i>						
<i>hops field, softplus, $t_0=2$</i>						
<i>delay field, relu, $t_0=0.5$</i>						
<i>delay field, relu, $t_0=2$</i>						
<i>delay field, softplus, $t_0=0.5$</i>						
<i>delay field, softplus, $t_0=2$</i>						
	264	-0.0726	[-0.1156, -0.0258]	0.180	0.039	0.820
	1584	-0.0120	[-0.0818, +0.0458]	1.000	1.000	1.000
	3168	+0.0214	[-0.0211, +0.0631]	0.289	0.383	1.000
	264	-0.0624	[-0.1052, -0.0252]	0.008	0.008	0.188
	1584	+0.0660	[-0.0015, +0.1277]	0.453	0.109	1.000
	3168	+0.0347	[+0.0016, +0.0873]	0.625	0.250	1.000
	264	-0.0103	[-0.0189, -0.0017]	0.754	0.105	1.000
	1584	+0.0200	[-0.0254, +0.0574]	0.289	0.641	1.000
	3168	+0.0052	[-0.0692, +0.0641]	1.000	0.812	1.000
	264	-0.0117	[-0.0214, -0.0018]	0.727	0.109	1.000
	1584	+0.0377	[-0.0238, +0.0861]	0.688	0.312	1.000
	3168	+0.0269	[-0.0094, +0.0637]	0.453	0.297	1.000
	264	-0.0273	[-0.0373, -0.0181]	0.016	0.016	0.359
	1584	-0.0139	[-0.0290, -0.0001]	0.754	0.232	1.000
	3168	+0.0141	[-0.0117, +0.0418]	1.000	0.492	1.000
	264	-0.0251	[-0.0446, -0.0069]	0.289	0.109	1.000
	1584	+0.0007	[-0.0037, +0.0053]	0.754	0.695	1.000
	3168	+0.0178	[-0.0054, +0.0434]	1.000	0.359	1.000
	264	-0.0033	[-0.0076, +0.0012]	1.000	0.359	1.000
	1584	-0.0081	[-0.0213, +0.0032]	0.344	0.193	1.000
	3168	+0.0274	[-0.0125, +0.0813]	1.000	0.438	1.000
	264	-0.0082	[-0.0138, -0.0023]	0.109	0.037	0.816
	1584	-0.0023	[-0.0102, +0.0049]	1.000	0.910	1.000
	3168	-0.0011	[-0.0260, +0.0235]	1.000	1.000	1.000

breaks the count down by operator and records whether each excluded run had escaped the constant solution before collapsing: the GCN collapses in 36/240 runs (12 of them escapers), the heat kernel in 24/240 (7) and the walk kernel in 27/240 (10). The instability is not specific to the spectral operators or to the learned t , and because most excluded runs had not escaped, exclusion removes non-escaping runs preferentially and every surviving mean is biased toward the escapers; the GCN, with the highest exclusion rate, is biased most. The paired tests of Section 5.3 are computed on the seeds valid for both kernels, so a seed removed for one kernel is removed for both. A fixed learning rate that is adequate for 200 epochs is not stable for 1000 on these graphs; we report the rate rather than tune it away.

5.7. Cost, and the scale at which the method is feasible

Table 17 reports the median wall-clock time per run on one consumer GPU with 24 GB of memory, with the eigendecomposition of each instance cached. The spectral operators build an $N \times N$ propagator per layer per instance per epoch from the eigenbasis, an $O(N^3)$ product that dominates at scale; the walk kernel builds two such products, for the real and imaginary parts, and

Table 14: The same runs scored on two metrics. For each arm-level cell, the seed votes heat:walk on MAE and on far-node error, how many votes moved, and the majority verdicts on the two metrics.

task	N	t arm	MAE heat:walk	far-node heat:walk	moved	verdict MAE / far-node
hops	264	relu, $t_0=0.5$	8:1	7:2	1	heat / heat
hops	264	relu, $t_0=2$	8:0	8:0	0	heat / heat
hops	264	softplus, $t_0=0.5$	7:3	6:4	1	heat / tie
hops	264	softplus, $t_0=2$	7:1	5:3	2	heat / tie
hops	1584	relu, $t_0=0.5$	4:2	3:3	1	n<7 / floor
hops	1584	relu, $t_0=2$	3:4	2:5	1	tie / tie
hops	1584	softplus, $t_0=0.5$	2:6	2:6	0	tie / tie
hops	1584	softplus, $t_0=2$	2:4	2:4	0	n<7 / n<7
hops	3168	relu, $t_0=0.5$	5:3	2:6	3	tie / floor
hops	3168	relu, $t_0=2$	3:1	1:3	2	n<7 / floor
hops	3168	softplus, $t_0=0.5$	2:3	2:3	0	n<7 / floor
hops	3168	softplus, $t_0=2$	5:2	2:5	3	tie / floor
delay	264	relu, $t_0=0.5$	7:0	7:0	0	heat / heat
delay	264	relu, $t_0=2$	7:1	6:2	1	heat / tie
delay	264	softplus, $t_0=0.5$	7:2	5:4	2	heat / tie
delay	264	softplus, $t_0=2$	9:1	8:2	1	heat / heat
delay	1584	relu, $t_0=0.5$	8:2	6:4	2	heat / floor
delay	1584	relu, $t_0=2$	8:2	6:4	2	heat / floor
delay	1584	softplus, $t_0=0.5$	8:2	7:3	1	heat / floor
delay	1584	softplus, $t_0=2$	4:5	5:4	1	tie / floor
delay	3168	relu, $t_0=0.5$	6:4	5:5	1	tie / floor
delay	3168	relu, $t_0=2$	4:5	4:5	0	tie / floor
delay	3168	softplus, $t_0=0.5$	1:5	3:3	2	n<7 / floor
delay	3168	softplus, $t_0=2$	3:6	5:4	2	tie / floor

Table 15: Sensitivity of the rule-dependent counts to the floor threshold g and the majority threshold. Left: task-scale cells of grid 2 in which the heat / walk kernel wins in every arm under rule 4 at the given majority. Right: hop-field cells of grid 1 at 1584 satellites and above that are rankable at the given floor.

floor g	majority $\geq 6/10$	$\geq 7/10$	$\geq 8/10$	rankable cells (grid 1, ≥ 1584 , hop)
0.01	2 heat / 0 walk	2 heat / 0 walk	0 heat / 0 walk	5 of 24
0.02	2 heat / 0 walk	2 heat / 0 walk	0 heat / 0 walk	2 of 24
0.05	2 heat / 0 walk	2 heat / 0 walk	0 heat / 0 walk	0 of 24
0.10	2 heat / 0 walk	2 heat / 0 walk	0 heat / 0 walk	0 of 24

squares them. On the 4400-satellite shell at 200 epochs a walk run takes 169 s against 81 s for the heat kernel and 21 s for the GCN, whose propagator is sparse. On CPU, before caching, the same cell took 1096 s, which is why the preliminary version stopped at 1584 satellites. As implemented, with dense $N \times N$ single-precision matrices for the eigenbasis, the propagator and their gradients, the method fits in 24 GB up to roughly $N = 19300$ nodes; beyond that it needs a factorised or sparse propagator, which is a different implementation. The mega-constellations of the introduction lie at the edge of that range and beyond it. For a comparison across scales, the cost of the operator is part of what is being compared.

Table 16: Runs excluded as pathologies (loss rising by more than 1% between thirds) out of runs, 1000 epochs, submitted capacity, both tasks and all t arms pooled, and whether the excluded run had escaped the constant solution (MAE below 90% of the constant predictor) before collapsing.

operator	excluded	of which had escaped	of which at floor	valid
GCN	36/240	12	24	204
heat	24/240	7	17	216
walk	27/240	10	17	213

Table 17: Median wall-clock seconds per training run on one GPU (eigendecomposition cached), submitted capacity, by shell and operator, at the two budgets. The 4400-satellite shell was not run at 1000 epochs.

N	200 epochs			1000 epochs		
	GCN	heat	walk	GCN	heat	walk
264	6	8	9	28	40	42
1584	8	10	11	30	42	45
3168	13	29	54	36	114	255
4400	21	81	169	—	—	—

6. Discussion

What was tested, and what was not. The object of this audit is the transition-probability kernel (3). It is a real nonnegative row-stochastic matrix, exactly as the heat kernel is, and applying it to features damps and mixes them as a diffusion step does; what it inherits from the walk is the shape of its rows, an oscillatory profile whose width grows linearly with t rather than as a square root (Figure 1). Once t is a trained parameter, the heat kernel reaches as far by learning a larger t , which the right panel of Figure 5 shows it does, in the first layer and in proportion to the shell. That the two kernels are then indistinguishable is what the operator definitions make plausible, and it is what the paired tests find. The argument of CTQW-GNN [12] for the unitary propagator in feature space, which damps no frequency and preserves feature norm, is a different object and is neither confirmed nor refuted here.

What the floor means for learned propagation on constellations. A budget that suffices on a 264-satellite shell does not on a 1584-satellite one, and the failure is silent: the run reports a finite, decreasing loss and an error that looks like a number. Comparisons of learned propagation across constellation sizes that hold the budget fixed will rank noise unless they carry the constant baseline beside every result.

What the seed-level bimodality and the vote noise mean. The split in Figure 4, with escape rates of 57% and 58% ($p = 0.95$), means that at the preliminary budget a majority verdict counts which seeds happened to escape. At the larger budget the majority statistic still moves by up to 3 seeds between two correlated metrics on the same runs (Section 5.4), which is the margin by which most of its verdicts were declared. The preliminary version’s 3-seed result at 1584 satellites sits inside the range that this bimodality produces; its direction is not confirmed at ten seeds and, under inference, no direction is.

Consequences for reporting. These are not new rules, and they are not the contribution; they are what this audit needed to avoid repeating its own preliminary version. Carry the constant predictor beside every result. Let the budget grow with the graph, or show that it need not. Treat any learned reach parameter’s parameterisation and initialisation as an arm. Use a paired test with a multiplicity correction, and state the false-positive rate of any vote rule that is reported beside it. Count collapsed runs and report whether they had escaped.

What would change the verdict. A propagator applied in feature space, unitary rather than stochastic, would test the argument of CTQW-GNN [12] directly; the wave-type models PDE-GCN [24] and graph-coupled oscillator networks [25] are the classical form of the same argument and were not run. A learning-rate schedule would reduce the collapse rate of Section 5.6 and might decide cells that are now undecided. A task with load, in which the field to be predicted depends on traffic and not only on geometry, and a time-varying evaluation that exercises the link churn the introduction invokes, would say whether any operator ranking survives contact with a network function. None of these is a small addition, and the present paper claims nothing about them.

7. Threats to validity

The two largest shells are synthetic, and all graphs are static. The 3168- and 4400-satellite graphs scale the Starlink shell-1 geometry and are not deployed constellations; they serve to grow the diameter. Every instance is an independent snapshot, so the time-varying property that motivates learned propagation on these graphs is never exercised.

The tasks are node fields, not network functions. Hop and delay fields isolate propagation from load, which is what the comparison needs, at the cost of distance from any routing or traffic-engineering objective. The findings bear on the evaluation of propagation operators, not on the routing or resource-management systems that might use them.

The floor threshold was chosen with the data in view. Rule 3’s value was set after the 4400-satellite cells were seen, as Section 4.2 states; Table 15 reports every dependent count at four values of it.

The majority rule has a 34.4% null false-positive rate. It is kept only so that the preliminary verdicts are reproducible, and no conclusion rests on it; the inference of rule 6 is what the paper stands on.

Exclusion is differential. Rule 2 removes runs after they have collapsed to the constant solution, so it removes non-escapers preferentially, and the GCN loses more runs than the spectral kernels (Table 16). Means are therefore optimistic, most for the GCN.

One learning rate, one optimiser, six evaluation instances per seed. The collapse rate is specific to Adam at 0.01 for 1000 epochs. Each seed evaluates on its own six instances, so the sixty instances of a cell are distinct, but instance-level dispersion within a seed is not reported.

The eigenbasis is cached in single precision. Training already casts the eigenpairs to single precision, so caching them is numerically identical to recomputing, and the cached and recomputed runs of a test cell agreed to 16 significant digits; but every arm shares one eigendecomposition per instance, and a numerical error in it would affect all arms alike.

Several references are preprints. CTQW-GNN [12], CTQWformer [13] and the LEO graph-learning work of Gu et al. [4] are unrefereed at the time of writing, and GraphQWalk [14] is a recent journal article. The argument this audit declines to test rests on the first of these; the audit itself rests on the preliminary version, which is not in the refereed record either.

The preliminary artifact does not reproduce its own headline. The released results of the preliminary version record 3301 parameters for the 1584-satellite experiment, which corresponds to the width-32, four-layer network its text describes, while the released code defaults to width 16 and three layers (612 parameters) and contains no script for that experiment. The preliminary version also ran without the seam cut that its text states. The present study fixes the seam cut on, runs both capacities, and releases the script for every number.

8. Conclusion

We audited a preliminary result of our own: that the transition-probability kernel of a continuous-time quantum walk outperformed the heat kernel on a 1584-satellite ISL graph and did so increasingly with scale. The result does not reproduce. Under the preliminary budget the large shells are a floor on which no operator learns; at the one shell where learning occurs the ranking follows the parameterisation and initialisation of the learned propagation time through a seed-level bimodality with indistinguishable escape rates; and with a budget that clears the floor, paired tests across ten seeds separate the two kernels in no arm-level cell after multiplicity correction, on either metric. The two kernels are the same kind of object, a real row-stochastic propagator with a learned reach, and they behave like it; what the learned reach buys is visible against fixed-reach decoupled propagation, which the better kernel in its best arm beats by 53% at 1584 satellites and 25% at 3168 on the hop field, and which its worst arm barely beats or does not. What the audit adds beyond the non-reproduction is measured: a silent budget floor that scales with the graph, a propagation-time parameterisation that decides rankings at small budgets, a seed-vote statistic whose noise amplitude is two to three seeds on these data, and a collapse rate of a tenth of long runs. Each is a reason to distrust a small-seed operator comparison on constellation graphs, including the one we published first.

Data and code availability

All code, the cached graph instances’ construction, every result file of the 1920 plus 720 runs, the scripts that produce every figure, table and number in this paper, and the dated protocol declarations are available at <https://github.com/haodpsut/leo-propagation-reach>. The preliminary version and its released artifact are available at <https://github.com/haodpsut/qi-gnn-dynamic-wireless>.

CRediT authorship contribution statement

Phuc Hao Do: Conceptualization, Methodology, Software, Investigation, Data curation, Writing, Visualization.

Declaration of competing interest

The author declares no competing interests.

Declaration of generative AI use

During the preparation of this work the author used a large language model (Claude, Anthropic) to assist with code, data analysis scripts and editing of the text. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

- [1] D. Bhattacharjee, A. Singla, Network topology design at 27,000 km/hour, in: Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies, 2019, pp. 341–354. doi:10.1145/3359989.3365407.
- [2] I. del Portillo, B. G. Cameron, E. F. Crawley, A technical comparison of three low earth orbit satellite constellation systems to provide global broadband, *Acta Astronautica* 159 (2019) 123–135. doi:10.1016/j.actaastro.2019.03.040.
- [3] H. Zhang, Z. Wang, S. Zhang, Q. Meng, H. Luo, Link-identified routing architecture in space, *IEEE Transactions on Network Science and Engineering* 12 (1) (2025) 392–408. doi:10.1109/tnse.2024.3498042.
- [4] Z. Gu, J. Choi, T. Q. S. Quek, J. Park, Duality-guided graph learning for real-time joint connectivity and routing in leo mega-constellations, *arXiv preprint arXiv:2601.21921* (2026). arXiv:2601.21921.
- [5] P. Maurya, A. Ramírez-Arroyo, T. B. Sørensen, S. B. Damsgaard, Machine learning-driven cellular-satellite multi-connectivity for monitoring livestock transport in rural areas, *Computer Communications* 246 (2026) 108366. doi:10.1016/j.comcom.2025.108366.
- [6] T. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, *arXiv preprint arXiv:1609.02907* ArXiv (Cornell University) (2016). arXiv:1609.02907, doi:10.48550/arxiv.1609.02907.
- [7] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, P. Vandergheynst, The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains, *IEEE Signal Processing Magazine* 30 (3) (2013) 83–98. doi:10.1109/MSP.2012.2235192.
- [8] B. Chamberlain, J. Rowbottom, M. I. Gorinova, S. Webb, E. Rossi, M. M. Bronstein, Grand: Graph neural diffusion, *arXiv preprint arXiv:2106.10934* ArXiv (Cornell University) (2021). arXiv:2106.10934, doi:10.48550/arxiv.2106.10934.
- [9] E. Farhi, S. Gutmann, Quantum computation and decision trees, *Physical Review A* 58 (2) (1998) 915–928. doi:10.1103/PhysRevA.58.915.
- [10] S. Dernbach, A. Mohseni-Kabir, S. Pal, M. Gepner, D. Towsley, Quantum walk neural networks with feature dependent coins, *Applied Network Science* 4 (1) (2019). doi:10.1007/s41109-019-0188-2.

- [11] G. Verdon, T. McCourt, E. Luzhnica, V. Singh, S. Leichenauer, J. D. Hidary, Quantum graph neural networks, arXiv preprint arXiv:1909.12264ArXiv (Cornell University) (2019). **arXiv:** 1909.12264, doi:10.48550/arxiv.1909.12264.
- [12] Y. Zhan, Z. Gao, J. Li, Y. Liu, H. sun, Continuous-time quantum walks based graph neural network, arXiv preprint arXiv:2608.20738 (2026). **arXiv:** 2608.20738.
- [13] Z.-L. Li, W. Yu, Y. Wu, C. Wang, Ctqwformer: A ctqw-based transformer for graph classification, arXiv preprint arXiv:2605.09486ArXiv.org (2026). **arXiv:** 2605.09486, doi: 10.48550/arXiv.2605.09486.
- [14] G. Liu, J. Zhao, H. Wei, Z. Zhou, Y. Song, X. Zhou, G. Qi, Graphqwalk: Learning structural node embeddings via continuous quantum walk, IEEE Transactions on Network Science and Engineering 13 (2026) 4779–4796. doi:10.1109/tNSE.2025.3644803.
- [15] M. Defferrard, X. Bresson, P. Vandergheynst, Convolutional neural networks on graphs with fast localized spectral filtering, arXiv preprint arXiv:1606.09375ArXiv (Cornell University) (2016). **arXiv:** 1606.09375, doi:10.48550/arxiv.1606.09375.
- [16] Q. Li, Z. Han, X.-M. Wu, Deeper insights into graph convolutional networks for semi-supervised learning, arXiv preprint arXiv:1801.07606ArXiv (Cornell University) (2018). **arXiv:** 1801.07606, doi:10.48550/arxiv.1801.07606.
- [17] K. Oono, T. Suzuki, Graph neural networks exponentially lose expressive power for node classification, arXiv preprint arXiv:1905.10947ArXiv (Cornell University) (2019). **arXiv:** 1905.10947, doi:10.48550/arxiv.1905.10947.
- [18] T. K. Rusch, M. M. Bronstein, S. Mishra, A survey on oversmoothing in graph neural networks, arXiv preprint arXiv:2303.10993ArXiv (Cornell University) (2023). **arXiv:** 2303.10993, doi: 10.48550/arxiv.2303.10993.
- [19] F. Wu, T. Zhang, A. H. Souza, C. Fifty, T. Yu, K. Q. Weinberger, Simplifying graph convolutional networks, arXiv preprint arXiv:1902.07153ArXiv (Cornell University) (2019). **arXiv:** 1902.07153, doi:10.48550/arxiv.1902.07153.
- [20] J. Gasteiger, A. Bojchevski, S. Günnemann, Predict then propagate: Graph neural networks meet personalized pagerank, arXiv preprint arXiv:1810.05997ArXiv (Cornell University) (2018). **arXiv:** 1810.05997, doi:10.48550/arxiv.1810.05997.
- [21] M. Chen, Z. Wei, Z. Huang, B. Ding, Y. Li, Simple and deep graph convolutional networks, arXiv preprint arXiv:2007.02133ArXiv (Cornell University) (2020). **arXiv:** 2007.02133, doi: 10.48550/arxiv.2007.02133.
- [22] U. Alon, E. Yahav, On the bottleneck of graph neural networks and its practical implications, arXiv preprint arXiv:2006.05205ArXiv (Cornell University) (2020). **arXiv:** 2006.05205, doi: 10.48550/arxiv.2006.05205.
- [23] J. Topping, F. D. Giovanni, B. P. Chamberlain, X. Dong, M. M. Bronstein, Understanding over-squashing and bottlenecks on graphs via curvature, arXiv preprint arXiv:2111.14522ArXiv (Cornell University) (2021). **arXiv:** 2111.14522, doi:10.48550/arxiv.2111.14522.

- [24] M. Eliasof, E. Haber, E. Treister, Pde-gcn: Novel architectures for graph neural networks motivated by partial differential equations, arXiv preprint arXiv:2108.01938 Neural Information Processing Systems (2021). [arXiv:2108.01938](#).
- [25] T. K. Rusch, B. P. Chamberlain, J. Rowbottom, S. Mishra, M. M. Bronstein, Graph-coupled oscillator networks, arXiv preprint arXiv:2202.02296 (2022). [arXiv:2202.02296](#).
- [26] J. Kempe, Quantum random walks: An introductory overview, Contemporary Physics 44 (4) (2003) 307–327. [doi:10.1080/00107151031000110776](#).
- [27] O. Mülken, A. Blumen, Continuous-time quantum walks: Models for coherent transport on complex networks, Physics Reports 502 (2-3) (2011) 37–87. [doi:10.1016/j.physrep.2011.01.002](#).
- [28] A. M. Childs, Universal computation by quantum walk, Physical Review Letters 102 (18) (2009). [doi:10.1103/PhysRevLett.102.180501](#).
- [29] W. Liang, F. Yan, A. M. Ilyasu, A. S. Salama, K. Hirota, A hadamard walk model and its application in identification of important edges in complex networks, Computer Communications 193 (2022) 378–387. [doi:10.1016/j.comcom.2022.07.045](#).
- [30] M. Handley, Delay is not an option, in: Proceedings of the 17th ACM Workshop on Hot Topics in Networks, 2018, pp. 85–91. [doi:10.1145/3286062.3286075](#).
- [31] S. Kassing, D. Bhattacharjee, A. B. Águas, J. E. Saethre, A. Singla, Exploring the "internet from space" with hypatia, in: Proceedings of the ACM Internet Measurement Conference, 2020, pp. 214–229. [doi:10.1145/3419394.3423635](#).
- [32] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, arXiv preprint arXiv:1811.05868 (2018). [arXiv:1811.05868](#).
- [33] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger, Deep reinforcement learning that matters, arXiv preprint arXiv:1709.06560 AAAI Conference on Artificial Intelligence (2017). [arXiv:1709.06560](#), [doi:10.1609/aaai.v32i1.11694](#).
- [34] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Sepah, E. Raff, K. Madan, V. Voleti, S. E. Kahou, V. Michalski, D. Serdyuk, T. Arbel, C. Pal, G. Varoquaux, P. Vincent, Accounting for variance in machine learning benchmarks, arXiv preprint arXiv:2103.03098 ArXiv (Cornell University) (2021). [arXiv:2103.03098](#), [doi:10.48550/arxiv.2103.03098](#).
- [35] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché-Buc, E. B. Fox, H. Larochelle, Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program), arXiv preprint arXiv:2003.12206 ArXiv (Cornell University) (2020). [arXiv:2003.12206](#), [doi:10.48550/arxiv.2003.12206](#).
- [36] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, arXiv preprint arXiv:1412.6980 UvA-DARE (University of Amsterdam) (2014). [arXiv:1412.6980](#), [doi:10.48550/arxiv.1412.6980](#).